Find the Promos — screenplay in, ranked promo list out

Praveen Roj · Enrolment 23117103 · Mechanical Engineering, IIT Roorkee · praveen_r@me.iitr.ac.in · Problem 1 · App: https://kuku-assignment.vercel.app/ · Code: https://github.com/iroj-09/KUKU-Assignment · Output: ranked_promos.csv (60 promos, in the submission file)

1. Why Problem 1, and how I scoped it

It is a weekly bottleneck (an editor watches every episode of every new show), it comes with real data (50 episodes), and its output is checkable: an episode, start/end times, a hook, a cliffhanger. I scoped it to **one step of the editor's work: producing a ranked shortlist of candidate promos**. The editor still watches and cuts; the tool decides where to look first.

2. Assumptions (the brief was silent)

- The screenplay timestamps are my only evidence. "First 3 s / last 5 s" are measured in screenplay seconds; I cannot see acting, visuals or music.
- A promo is **one continuous 20–60 s stretch that starts and ends on event boundaries**, so no line is cut in half. Promos that join moments or span episodes are not built (the output format already stores a list of segments).
- Up to 12 candidates per episode reach de-duplication; at most 3 promos per episode survive; two clips "overlap" when the shared time is at least half of the shorter one.
- The 5 model score weights (hook .25, cliffhanger .25, emotion .20, stand-alone .15, progression .15) and the 70% model / 30% heuristic blend are my judgement, not learned from data.

3. How it works — what is code, what is AI, where a human stays

Code (free, deterministic)	Parse screenplays → build every valid window (**53,182** windows in 16 s) → cheap keyword/cadence score → keep best 12 per episode (600) → drop overlaps (**60**) → cut out each clip's first 3 s and last 5 s as evidence → **re-check every timestamp the model returns** against those windows → rank.
AI (one job)	For the 60 shortlisted clips only: find the hook and the cliffhanger *inside the evidence windows*, score hook / cliffhanger / stakes / stand-alone clarity / progression, write one line on why it works, list risks. JSON is schema-validated; an out-of-window or missing timestamp is never trusted, it downgrades the clip to PARTIAL/FAIL.
Human	Review app (approve / reject / notes, backup + import); every shortlisted clip is a candidate for a person to approve or reject.

Re-runnable: `python run_all.py --input <folder>` runs all 8 steps (`--no-llm` works with no API key). **Model is swappable** by environment variable (Gemini, any OpenAI-compatible API, Anthropic); a test runs the whole evaluator against a fake non-Gemini server.

4. Result on the 50-episode pack

53,182 windows → 600 → **60 promos covering all 50 episodes**; timestamp check 46 PASS / 14 PARTIAL / 0 FAIL. Ranked first: Ep 32 (107.0–150.8 s: *"And what is wrong with that?!"* → *"...let's see how brave you are when it touches your skin!"*), Ep 24 (33.8–89.8 s), Ep 45 (42.0–88.2 s). Every promo carries *why it works* and *why ranked here* (score breakdown, timestamp status, main risk) in `promos_submission.md/.csv/.json`.

5. Scale and cost

The expensive judgement is spent on ~0.1% of windows: a whole 50-episode show is **12 requests, about 68k input + 13k output tokens**, so the free tier was enough, and cost grows with the shortlist, not episode length. For many shows: one run per show (16 s of CPU), results cached by clip id (resumable), and a stronger model can re-judge only each show's top ~20. Paid API use I would add: a pairwise "which would you post first?" pass on those top 20.

6. How I know it is good — and where it fails (measured: `quality_report.md`)

- **My first version was wrong.** It used fixed 5-second steps: every start/end was a multiple of 5 s (mid-line cuts) and 46% of clips ran 55 s or more. Rebuilt on event boundaries: long clips 18%, mean model score 70.6 → 75.7, no FAIL.
- **Little choice:** 40 of 50 episodes get only 1 promo (clips from one scene overlap and get pruned).
- **14 of 60 are PARTIAL:** the model quoted a strong line that is not in the first 3 s / last 5 s (10× cliffhanger, 4× hook).
- **The heuristic barely agrees with the model** (correlation 0.08), so its 30% share is mostly noise; it is one setting (`LOCAL_WEIGHT`) to change once labels exist.
- **The model is a lenient judge:** 58 of 60 "promising", scores only spread ±7.6, so ranks near each other are not meaningfully different.
- **Needs earlier episodes:** 12 of 60 list this as a risk; mean stand-alone score falls from 8.2 (early) to 7.6 (late episodes).

Note: the numbers above come from the pipeline itself (not from a manual accuracy check), so they show where the tool is weak, not how often it is right.

7. AI tools I used — and where they got it wrong

Claude Code (built and refactored the pipeline, wrote tests), **Gemini 3.5 Flash-Lite** (the judge inside the pipeline). **Where it was wrong:** the AI-written docs and my earlier results claimed "beat-aligned windows", but the saved outputs actually came from the 5-second sliding windows. Nothing flagged it; I only noticed when a fresh run shared **0 of 74** clips with the old one. Fix: regenerate everything, archive the old run, and add tests for the parameters that drifted. Also: Gemini as judge praises almost everything, so its score is one input to the ranking, not the final word.

Note on the deployed app: it ships no screenplay data (confidential), so for a new upload it runs a lightweight in-browser heuristic (3 clips per script by default) and the optional AI pass. The full pipeline result for the input pack (60 promos with heuristic and model

scores) is `ranked_promos.csv`.

8. Next

Label ~50 clips with an editor and fit the weights; build joined and cross-episode promos; pick shortlists for variety so each episode offers 2–3 options; run the cinematic cues (acting, pacing) through video once available.